Tree DSA Handbook

A Complete, Practical Guide to Tree Data Structures & Algorithms in Python

PART 1: FUNDAMENTALS

1.1 What is a Tree?

A tree is a hierarchical, connected, acyclic structure made of nodes, where one node is designated the **root** and every other node has exactly one **parent**, reachable by exactly one path from the root. Trees are a special case of graphs (connected, undirected, acyclic — see the Graph handbook, 1.3) but are important enough, and recursive enough in structure, to deserve their own dedicated toolkit. A tree with n nodes always has exactly n-1 edges.

1.2 Core Terminology

Root	The single topmost node with no parent; every other node is reachable from it.
Parent / Child	A node directly connected one level up (parent) or one level down (child) from another.
Leaf	A node with no children.
Sibling	Nodes sharing the same parent.
Ancestor / Descendant	Any node on the path from a node up to the root (ancestor), or any node reachable going down from it (descendant).
Subtree	A node together with all of its descendants, itself forming a valid tree.
Depth of a Node	Number of edges from the root down to that node (root has depth 0).
Height of a Node	Number of edges on the longest downward path from that node to a leaf (a leaf has height 0).
Height of a Tree	Height of the root — the longest root-to-leaf path.
Degree of a Node	Number of children it has.
Binary Tree	A tree where every node has at most 2 children, conventionally called left and right.
Binary Search Tree (BST)	A binary tree where every node's left subtree contains only smaller values and right subtree only larger values.

1.3 Types of Trees

By Shape / Fill Pattern

- **Full Binary Tree** — every node has either 0 or 2 children (never exactly 1).
- **Complete Binary Tree** — every level is fully filled except possibly the last, which fills left to right (the shape used by array-backed heaps).
- **Perfect Binary Tree** — all internal nodes have 2 children AND all leaves are at the same depth.
- **Balanced Binary Tree** — height difference between left and right subtrees of every node is bounded (e.g., ≤ 1 for AVL) — guarantees O(log n) height.
- **Degenerate / Skewed Tree** — each node has only one child, effectively a linked list; height = n-1, worst case for most tree algorithms.

By Ordering / Purpose

- **Binary Search Tree (BST)** — ordered for O(h) search/insert/delete.
- **Self-Balancing BST (AVL, Red-Black)** — a BST that rebalances itself on insert/delete to guarantee O(log n) height.
- **Heap** — a complete binary tree satisfying the heap-order property (parent ≤ or ≥ children); typically array-backed, not pointer-backed.
- **Trie (Prefix Tree)** — an n-ary tree where each path from root spells out a string/prefix; not value-ordered like a BST.
- **N-ary Tree** — each node may have any number of children (not limited to 2), e.g., file systems, org charts.
- **Segment Tree / Fenwick Tree** — specialized trees for efficient range queries/updates over an array (mentioned in the cheat sheet; a deep dive belongs in a dedicated advanced-structures handbook).

1.4 Ways to Represent a Tree

Node + Pointers (most common)	Each node object stores its value and references to its children (left/right for binary, a list for N-ary). Natural for recursion; used throughout this handbook.
Array Representation	Used for complete binary trees (heaps): node at index i has children at 2i+1 and 2i+2, and parent at (i-1)//2. Space-efficient, no pointers needed, but only works cleanly when the tree is complete.
Parent Pointer Array	parent[i] = index of i’s parent; efficient for answering "is X an ancestor of Y" style queries and for Union-Find-style processing, but awkward for top-down traversal (no direct child access).
Left-Child Right-Sibling	A trick to represent N-ary trees using only 2 pointers per node (first child, next sibling) — turns any N-ary tree into an equivalent binary-tree-shaped structure.

1.5 How to Read This Handbook

For every topic below you will get, in this exact order: (1) What it’s for, (2) Use Cases, (3) Limitations, (4) Time & Space Complexity, (5) Approach (short) with pseudo-code where helpful, (6) Keywords & Constraints, (7) Fully commented Python code, and (8) 10 Practice Questions each with a short reasoning of why the technique fits.

PART 2: PATTERNS & ALGORITHMS

2.1 Tree Traversals — Preorder, Inorder, Postorder

What it's for: The three classic depth-first ways to visit every node of a binary tree exactly once, differing only in WHEN the current node is "visited" relative to recursing into its left and right children. These three orders are the foundation nearly every other tree algorithm builds on.

Use Cases:

- Preorder — copying/cloning a tree, serializing a tree (root info comes before children, easy to rebuild).
- Inorder — retrieving BST values in SORTED order (this is the single most important fact about inorder traversal).
- Postorder — safely deleting/freeing a tree (children before parent), evaluating expression trees, computing subtree-dependent values bottom-up.
- All three are the building blocks for height, diameter, path-sum, and virtually every recursive tree algorithm in this handbook.

Limitations:

- Recursive versions use O(h) call-stack space and can hit Python's recursion limit on a very skewed (linked-list-like) tree.
- None of the 3 DFS orders visit nodes level-by-level — use Level Order Traversal (2.2) when "breadth" or "distance from root" matters.
- Iterative versions require an explicit stack and careful bookkeeping (especially inorder and postorder) — easy to get the push/pop order wrong.

Time Complexity: O(n) — every node visited exactly once **Space Complexity:** O(h) recursion/explicit-stack space (h = tree height); O(n) worst case for a skewed tree

Approach (short): PSEUDOCODE (preorder): visit(node) -> process(node); visit(node.left); visit(node.right). Inorder swaps the process() to the middle; postorder moves it to the end. Each is a one-line reorder of the same 3 statements.

Keywords & Constraints: "traverse the tree", "visit every node", "sorted order from a BST" (inorder), "process children before parent" (postorder), n ≤ 10^5 nodes typical.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def preorder_recursive(node, result=None):
    """Visit ROOT, then left subtree, then right subtree."""
    if result is None:
        result = []
    if node is None:
        return result
    result.append(node.val)                # 1. process root FIRST
    preorder_recursive(node.left, result)   # 2. then left subtree
    preorder_recursive(node.right, result)  # 3. then right subtree
    return result


def inorder_recursive(node, result=None):
    """Visit left subtree, then ROOT, then right subtree -- gives SORTED order for a BST."""
    if result is None:
        result = []
    if node is None:
        return result
    inorder_recursive(node.left, result)
    result.append(node.val)                 # process root in the MIDDLE
```

```
    inorder_recursive(node.right, result)
    return result


def postorder_recursive(node, result=None):
    """Visit left subtree, then right subtree, then ROOT -- children processed before parent."""
    if result is None:
        result = []
    if node is None:
        return result
    postorder_recursive(node.left, result)
    postorder_recursive(node.right, result)
    result.append(node.val)                  # process root LAST
    return result


def preorder_iterative(root):
    """Iterative preorder using an explicit stack (push right child before left -> left pops first)."""
    if root is None:
        return []
    result, stack = [], [root]
    while stack:
        node = stack.pop()
        result.append(node.val)
        if node.right:
            stack.append(node.right)          # push right FIRST
        if node.left:
            stack.append(node.left)           # push left LAST so it's popped/processed first
    return result


def inorder_iterative(root):
    """Iterative inorder: walk left as far as possible, pushing nodes, then process and go right."""
    result, stack = [], []
    node = root
    while stack or node:
        while node:                           # go as far left as possible, remembering the path
            stack.append(node)
            node = node.left
        node = stack.pop()                    # backtrack to the deepest unprocessed ancestor
        result.append(node.val)
        node = node.right                     # then explore its right subtree
    return result


def postorder_iterative(root):
    """Iterative postorder via a reversed modified-preorder trick (root,right,left) reversed = (left,right,root)."
    if root is None:
        return []
    result, stack = [], [root]
    while stack:
        node = stack.pop()
        result.append(node.val)
        if node.left:
            stack.append(node.left)           # push left first this time (opposite of preorder)
        if node.right:
            stack.append(node.right)
    return result[::-1]                       # reverse (root,right,left) -> (left,right,root)
```

Practice Questions — Tree Traversals — Preorder, Inorder, Postorder

Q1. Preorder Traversal (Recursive) Return the preorder traversal of a binary tree. *Approach:* Process the node itself before recursing left then right — the definition of preorder, directly translated to code.

Q2. Inorder Traversal (Recursive) Return the inorder traversal of a binary tree. *Approach:* Recurse left, process the node, recurse right — for a BST this produces values in sorted ascending order, which is the key reason inorder matters.

Q3. Postorder Traversal (Recursive) Return the postorder traversal of a binary tree. *Approach:* Recurse left, recurse right, then process the node last — needed whenever children must be fully handled before the parent (e.g., deleting a tree, evaluating an expression tree).

Q4. Preorder Traversal (Iterative) Return preorder traversal without recursion. *Approach:* Use an explicit stack, pushing right child before left so the left child is popped and processed first, mimicking the recursive call order.

Q5. Inorder Traversal (Iterative) Return inorder traversal without recursion. *Approach:* Push every left descendant onto the stack first; once you can't go left anymore, pop, process, then move to the popped node's right child and repeat.

Q6. Postorder Traversal (Iterative) Return postorder traversal without recursion. *Approach:* Run a modified preorder that visits (root, right, left) using a stack, then reverse the result to get (left, right, root) — avoids the more complex two-stack or visited-flag approaches.

Q7. Binary Tree Postorder for Expression Tree Evaluation Evaluate an arithmetic expression tree (leaves = operands, internal nodes = operators). *Approach:* Postorder guarantees both operands (left and right subtree results) are fully computed before applying the operator at the current node.

Q8. Print Boundary of a Binary Tree Print left boundary, leaves, and right boundary of a binary tree in order. *Approach:* Combine three traversal passes: a top-down left-edge walk, an inorder-style leaf collection, and a bottom-up right-edge walk — shows how the 3 base traversals compose into more complex outputs.

Q9. Vertical Order Traversal Group node values by their horizontal distance from the root. *Approach:* A preorder-style DFS (or BFS) that tracks a running horizontal-distance offset per node, bucketing values by that offset into a dictionary/sorted structure.

Q10. Construct String from Binary Tree (Preorder with Parentheses) Convert a tree to a string using preorder traversal with parentheses to disambiguate structure. *Approach:* Preorder-visit each node, wrapping each non-empty child's output in parentheses, and including an empty pair for a missing left child only when a right child exists — tests precise control over WHEN each piece is appended.

2.2 Level Order Traversal (BFS) & Variants

What it's for: Visits tree nodes level by level (breadth-first) using a queue, processing all nodes at depth d before any node at depth d+1. The tree analogue of graph BFS (see Graph handbook, 2.1).

Use Cases:

- Printing a tree level by level, or grouping node values by depth.
- Zigzag traversal, right-side/left-side view of a tree.
- Finding the minimum depth (shortest root-to-leaf path) — BFS finds this faster than DFS since it stops at the first leaf found.
- Level-average / level-sum queries, connecting same-level nodes (populate next-right pointers).

Limitations:

- Uses O(w) space where w is the tree's maximum width, which can be up to O(n/2) for a complete binary tree's last level — more memory-hungry than DFS's O(h) in a wide, shallow tree.
- Not suitable when you need to compute something bottom-up through children (use postorder DFS instead — see 2.5, 2.6).
- Naively re-scanning the queue to find level boundaries requires care (snapshot the queue's current length at the top of each level's loop).

Time Complexity: O(n) — every node enqueued and dequeued once **Space Complexity:** O(w) for the queue, where w is the maximum width of the tree (up to O(n) for a wide tree)

Approach (short): PSEUDOCODE: queue <- [root]; while queue not empty: level_size <- len(queue); for _ in range(level_size): pop front, process it, push its children; (the level_size snapshot is what separates one level from the next).

Keywords & Constraints: "level order", "level by level", "zigzag", "right side view", "minimum depth", n ≤ 10^5 nodes.

```
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def level_order(root):
    """Return node values grouped level by level."""
    if root is None:
        return []
    result, queue = [], deque([root])
    while queue:
        level_size = len(queue)              # SNAPSHOT: exactly how many nodes are in THIS level
        level_vals = []
        for _ in range(level_size):
            node = queue.popleft()
            level_vals.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level_vals)
    return result


def zigzag_level_order(root):
    """Level order, but alternate left-to-right / right-to-left each level."""
    if root is None:
        return []
    result, queue, left_to_right = [], deque([root]), True
    while queue:
```

```
        level_size = len(queue)
        level_vals = deque()
        for _ in range(level_size):
            node = queue.popleft()
            if left_to_right:
                level_vals.append(node.val)
            else:
                level_vals.appendleft(node.val)   # insert at front to reverse this level's order
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(list(level_vals))
        left_to_right = not left_to_right
    return result


def right_side_view(root):
    """The value of the LAST node visited at each level == what you'd see from the right."""
    if root is None:
        return []
    result, queue = [], deque([root])
    while queue:
        level_size = len(queue)
        for i in range(level_size):
            node = queue.popleft()
            if i == level_size - 1:              # last node processed in this level
                result.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return result


def min_depth(root):
    """Minimum root-to-leaf depth -- BFS stops at the FIRST leaf, faster than DFS in wide trees."""
    if root is None:
        return 0
    queue = deque([(root, 1)])
    while queue:
        node, depth = queue.popleft()
        if node.left is None and node.right is None:
            return depth                         # first leaf found = minimum depth (BFS guarantee)
        if node.left:
            queue.append((node.left, depth + 1))
        if node.right:
            queue.append((node.right, depth + 1))
```

Practice Questions — Level Order Traversal (BFS) & Variants

Q1. Binary Tree Level Order Traversal Return node values grouped by level. *Approach:* Snapshot the queue length at the start of each level's loop iteration so exactly that many nodes (this level only) are processed before moving to the next level.

Q2. Zigzag Level Order Traversal Return level order traversal alternating direction each level. *Approach:* Same level-order BFS skeleton, but append to the front or back of each level's result depending on a toggling left_to_right flag.

Q3. Binary Tree Right Side View Return the values visible from the right side of the tree. *Approach:* Run level order BFS and keep only the LAST node processed in each level's inner loop — that's the rightmost node at that depth.

Q4. Minimum Depth of Binary Tree Find the length of the shortest root-to-leaf path. *Approach:* BFS naturally finds the shortest path first — return as soon as the first leaf (node with no children) is dequeued, no need to explore deeper levels.

Q5. Average of Levels in Binary Tree Return the average value of nodes at each level. *Approach:* Standard level order BFS, summing values processed in each level's inner loop and dividing by that level's size before moving on.

Q6. Populating Next Right Pointers in Each Node Connect each node to its next right neighbor at the same level. *Approach:* Level order BFS where, instead of collecting values, you link consecutive nodes popped from the same level's batch, setting the last node's next pointer to None.

Q7. Cousins in Binary Tree Determine if two nodes are cousins (same depth, different parents). *Approach:* BFS level by level, tracking each node's parent alongside it in the queue; the two target values are cousins iff found at the same level with different parents.

Q8. N-ary Tree Level Order Traversal Return level order traversal of a tree where nodes can have any number of children. *Approach:* Identical level-order BFS skeleton, but the inner loop enqueues ALL of a node's children (a list) instead of just left/right.

Q9. Maximum Width of Binary Tree Find the maximum number of nodes across any level, counting nulls between the leftmost and rightmost non-null node. *Approach:* BFS while assigning each node a positional INDEX (as if the tree were a complete binary tree, index*2+1/+2 for children); width of a level = last index - first index + 1.

Q10. Binary Tree Level Order Traversal II (Bottom-Up) Return level order traversal from the bottom level up to the root level. *Approach:* Run standard level order BFS, then simply reverse the final list of levels — shows how a "variant" can sometimes just be a post-processing step on the base pattern.

2.3 BST Insertion & Search

What it's for: Uses the Binary Search Tree ordering property (left subtree < node < right subtree) to insert new values and search for existing ones in O(h) time by discarding half the remaining tree at every step, the same way binary search discards half an array.

Use Cases:

- Maintaining a dynamically-updated sorted collection supporting fast insert/search (unlike a sorted array, no O(n) shifting needed to insert).
- Implementing ordered maps/sets, range queries, and "find closest value" style lookups.
- Validating whether a tree satisfies the BST property.
- A foundational building block for self-balancing trees (2.9) and other ordered structures.

Limitations:

- Without balancing, a BST built from sorted/adversarial input degenerates into a linked list — O(n) height, making insert/search O(n) instead of the hoped-for O(log n).
- Duplicate values need an explicit convention (reject, allow on one side, or store a count) — the plain BST definition doesn't specify this.
- BST-ness is a GLOBAL property, not just "left child < node < right child" checked locally — a common bug is checking only immediate children instead of carrying a valid (low, high) range down the recursion.

Time Complexity: O(h) average O(log n) for a balanced tree; O(n) worst case for a skewed tree **Space Complexity:** O(h) recursion stack for recursive versions; O(1) for iterative versions

Approach (short): PSEUDOCODE (search): if root is None or root.val == target: return root; if target < root.val: recurse left; else: recurse right. Insertion follows the exact same path-finding logic, attaching the new node where a None child is found.

Keywords & Constraints: "binary search tree", "insert into BST", "search in BST", "validate BST", n ≤ 10^5 nodes.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def search_bst(root, target):
    """Find the node with value == target, or None if absent."""
    if root is None or root.val == target:
        return root
    if target < root.val:
        return search_bst(root.left, target)     # discard the right half of the tree entirely
    return search_bst(root.right, target)          # discard the left half entirely


def insert_bst(root, val):
    """Insert val into the BST, returning the (possibly new) root."""
    if root is None:
        return TreeNode(val)                       # found the correct empty spot -- create the new node here
    if val < root.val:
        root.left = insert_bst(root.left, val)      # recurse left, reattach the (possibly updated) subtree
    elif val > root.val:
        root.right = insert_bst(root.right, val)
    return root                                     # unchanged if val already exists (no duplicates)


def insert_bst_iterative(root, val):
    """Iterative version -- avoids recursion, same O(h) time."""
    new_node = TreeNode(val)
    if root is None:
        return new_node
    node = root
    while True:
```

```
            if val < node.val:
                if node.left is None:
                    node.left = new_node
                    break
                node = node.left
            elif val > node.val:
                if node.right is None:
                    node.right = new_node
                    break
                node = node.right
            else:
                break                                   # duplicate -- do nothing
        return root


    def is_valid_bst(root):
        """Check the GLOBAL BST property using a valid (low, high) range threaded down the recursion."""
        def helper(node, low, high):
            if node is None:
                return True
            if not (low < node.val < high):             # node must fit strictly within its inherited range
                return False
            return helper(node.left, low, node.val) and helper(node.right, node.val, high)
        return helper(root, float('-inf'), float('inf'))
```

Practice Questions — BST Insertion & Search

Q1. Search in a Binary Search Tree Find and return the subtree rooted at a target value. *Approach:* Compare target to the current node and go left or right accordingly, discarding half the remaining tree at every step — O(h) instead of O(n).

Q2. Insert into a Binary Search Tree Insert a new value while maintaining BST property. *Approach:* Walk down comparing the value to each node until you reach a None child, then attach the new node there.

Q3. Validate Binary Search Tree Check whether a binary tree satisfies the BST ordering property. *Approach:* Thread a valid (low, high) range down the recursion instead of only comparing to immediate children — a node deep in the left subtree must still be less than every ancestor above it, not just its direct parent.

Q4. Convert Sorted Array to BST (Balanced) Build a height-balanced BST from a sorted array. *Approach:* Pick the middle element as the root (mirroring binary search's midpoint), recursively build the left half from the left subarray and the right half from the right subarray.

Q5. Find the Closest Value in a BST Find the BST value closest to a given target. *Approach:* Walk down the tree comparing target to each node, updating a running "closest so far" answer at every step, moving left/right based on comparison — no backtracking needed due to BST ordering.

Q6. Kth Smallest Element in a BST Find the kth smallest value in a BST. *Approach:* Inorder traversal of a BST visits values in ascending order (see 2.1) — stop as soon as the kth value has been visited, no need to build the full sorted list.

Q7. Range Sum of BST Sum all node values within a given [low, high] range. *Approach:* Prune the recursion using BST ordering: skip the left subtree entirely if the current value ≤ low is false is irrelevant— skip left if val < low, skip right if val > high, avoiding a full O(n) traversal.

Q8. Two Sum IV — Input is a BST Determine if two nodes sum to a target value. *Approach:* Inorder traversal collects sorted values, then a two-pointer scan finds a pair summing to target — combines the BST-gives-sorted-order fact with the classic two-pointer technique.

Q9. Insert into a BST (Multiple Values, Bulk Build) Build a BST by inserting a stream/list of values one at a time. *Approach:* Repeatedly apply single-value BST insertion; demonstrates why input ORDER matters — inserting already-sorted values produces a degenerate, unbalanced tree.

Q10. Trim a Binary Search Tree Remove nodes outside a given [low, high] range while keeping it a valid BST.
Approach: Recursively trim: if a node's value is out of range, discard it and recurse into the ONE subtree that could still contain valid values; otherwise trim both children and keep the node.

2.4 BST Deletion

What it's for: Removes a node from a BST while preserving the BST property, handling 3 distinct structural cases: the node has no children, exactly one child, or two children. The two-children case is the tricky one and is the heart of this topic.

Use Cases:

- Any dynamic ordered-set/map implementation that needs to support removal.
- Database indexing structures (conceptually related), scheduling systems removing completed items from an ordered structure.
- Interview staple for testing whether you truly understand BST structure (not just insertion).

Limitations:

- Repeated deletions (especially always removing the same side's successor/predecessor) can gradually unbalance a plain BST — production systems use a self-balancing variant (2.9) to guarantee performance stays O(log n).
- Must correctly reattach subtrees — a common bug is losing the deleted node's remaining child/children instead of splicing them back in.
- Deleting by value assumes values are unique; duplicate-value BSTs need a defined convention for which instance gets removed.

Time Complexity: O(h) — O(log n) average for a balanced tree, O(n) worst case for a skewed tree **Space Complexity:** O(h) recursion stack

Approach (short): PSEUDOCODE: find the node (as in search). Case 1 (no children): simply remove it (return None to the parent). Case 2 (one child): replace the node with its single child. Case 3 (two children): find the INORDER SUCCESSOR (smallest value in the right subtree), copy that value into the current node, then recursively delete the successor from the right subtree (which now has at most one child, reducing to case 1 or 2).

Keywords & Constraints: "delete a node from BST", "remove from binary search tree", "inorder successor/predecessor", n ≤ 10^5 nodes.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def find_min(node):
    """Leftmost node = smallest value in this subtree (the inorder successor's source when going right first)."""
    while node.left:
        node = node.left
    return node


def delete_node(root, key):
    """Delete the node with value == key from the BST, returning the (possibly new) root."""
    if root is None:
        return None                                    # key not found -- nothing to delete

    if key < root.val:
        root.left = delete_node(root.left, key)         # target is in the left subtree, recurse there
    elif key > root.val:
        root.right = delete_node(root.right, key)       # target is in the right subtree, recurse there
    else:
        # FOUND the node to delete -- now handle the 3 cases:
        if root.left is None and root.right is None:
            return None                                  # Case 1: leaf -- just remove it
        if root.left is None:
            return root.right                             # Case 2: only right child -- replace with it
        if root.right is None:
            return root.left                              # Case 2: only left child -- replace with it

        # Case 3: two children -- find inorder successor (smallest in right subtree)
```

```
            successor = find_min(root.right)
            root.val = successor.val                          # copy successor's value into this node
            root.right = delete_node(root.right, successor.val)  # then delete the successor (now case 1 or 2)

        return root


    def delete_node_using_predecessor(root, key):
        """Alternative: use the inorder PREDECESSOR (largest in left subtree) instead of successor -- equally valid."""
        if root is None:
            return None
        if key < root.val:
            root.left = delete_node_using_predecessor(root.left, key)
        elif key > root.val:
            root.right = delete_node_using_predecessor(root.right, key)
        else:
            if root.left is None:
                return root.right
            if root.right is None:
                return root.left
            node = root.left
            while node.right:                                  # find largest value in left subtree
                node = node.right
            root.val = node.val
            root.left = delete_node_using_predecessor(root.left, node.val)
        return root
```

Practice Questions — BST Deletion

Q1. Delete Node in a BST Remove a node with a given key while maintaining BST validity. *Approach:* Handle all 3 cases explicitly: leaf (remove directly), one child (splice it up), two children (replace value with inorder successor, then delete the successor from the right subtree).

Q2. Delete Leaf Nodes with a Given Value Remove all leaf nodes matching a target value. *Approach:* Postorder traversal (process children before the node itself) so that removing a leaf can expose its former parent as a NEW leaf, which also gets checked/removed in the same pass.

Q3. Find Inorder Successor in a BST Find the node with the next larger value than a given node. *Approach:* If the node has a right child, the successor is the leftmost node in that right subtree; otherwise, it's the lowest ancestor for which the given node lies in the left subtree.

Q4. Find Inorder Predecessor in a BST Find the node with the next smaller value than a given node. *Approach:* Mirror image of successor-finding: if a left child exists, the predecessor is the rightmost node in that left subtree; otherwise walk up to the lowest ancestor where the node lies in the right subtree.

Q5. Delete a Range of Values from a BST Remove all nodes with values inside a given [low, high] range. *Approach:* Extend single-value deletion into a recursive trim: if a node's value falls inside the range, delete it (using the standard 3-case logic) and continue processing what takes its place; otherwise recurse into whichever subtree could contain in-range values.

Q6. Balance a BST After Many Deletions Given a BST degraded by repeated deletions, rebalance it. *Approach:* Inorder traversal extracts values in sorted order (already a sorted array), then rebuild via the same "pick the middle as root" recursive construction used for Convert Sorted Array to BST (2.3).

Q7. Remove All Nodes Which Don't Satisfy a Path Condition Delete leaf nodes/paths that don't meet a cumulative condition (e.g., path sum too small). *Approach:* Postorder recursion returns whether a subtree should survive; a node deletes a child by setting its pointer to None based on the child's returned survival signal.

Q8. Delete Node in a BST (Iterative Version) Perform BST deletion without recursion. *Approach:* Manually track the parent pointer while searching for the target, then splice the found node out by directly reassigning the parent's left/right pointer instead of relying on the recursive return-value pattern.

Q9. Two-Node Swap to Fix a BST Two nodes in a BST were swapped by mistake; find and fix them. *Approach:* Inorder traversal of a valid BST is strictly increasing; scanning the inorder sequence for out-of-order adjacent pairs identifies exactly which two node values need to be swapped back.

Q10. Merge Two BSTs Merge two separate BSTs into one balanced BST. *Approach:* Inorder-traverse both BSTs into two sorted lists, merge the two sorted lists (like Merge Sort's merge step), then build a balanced BST from the merged sorted array.

2.5 Height, Depth, Diameter & Balance Checking

What it's for: A family of postorder-based measurements describing a tree's shape: how tall it is, how far a given node sits from the root, the longest path between any two nodes, and whether every subtree is height-balanced.

Use Cases:

- Determining if a BST/tree is balanced enough to guarantee O(log n) operations.
- Diameter-style problems — finding the longest path in a tree (network latency worst-case, etc.).
- Depth-based problems — minimum/maximum depth, depth of a specific node, level-based aggregation.
- A prerequisite check before applying algorithms that assume/require a balanced tree.

Limitations:

- Diameter is commonly miscoded as "just the height difference at the root" — it must be checked at EVERY node, since the longest path might live entirely inside a subtree, not through the root.
- A naive balance check that recomputes height from scratch at every node is $O(n^2)$ in the worst case (skewed tree) — an efficient version combines height computation and balance checking into a single postorder pass (O(n)).
- Height and depth are easy to conflate; be precise about which direction you're measuring (down-to-leaf vs. down-from-root).

Time Complexity: O(n) for an efficient single-pass version; $O(n^2)$ worst case for a naive height-recomputing version
Space Complexity: O(h) recursion stack

Approach (short): PSEUDOCODE (height): height(node) = -1 if node is None else 1 + max(height(left), height(right)). For diameter/balance, compute height bottom-up (postorder) while SIMULTANEOUSLY updating a running answer (max diameter seen, or an "unbalanced" flag) at each node — one traversal does both jobs.

Keywords & Constraints: "height of tree", "diameter", "balanced binary tree", "depth of node", n ≤ 10^5 nodes.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def height(node):
    """Height in EDGES: empty tree = -1, single leaf = 0."""
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))


def depth_of_node(root, target, current_depth=0):
    """Depth of a target value from the root (root itself has depth 0)."""
    if root is None:
        return -1                                          # not found
    if root.val == target:
        return current_depth
    left_depth = depth_of_node(root.left, target, current_depth + 1)
    if left_depth != -1:
        return left_depth
    return depth_of_node(root.right, target, current_depth + 1)


def diameter(root):
    """Longest path between ANY two nodes (in edges) -- checked at every node, not just the root."""
    best = [0]
    def dfs(node):
        if node is None:
            return -1                                      # height of an empty subtree
        left_h = dfs(node.left)
        right_h = dfs(node.right)
        # candidate diameter THROUGH this node = edges down-left + edges down-right + the 2 edges connecting them
        best[0] = max(best[0], (left_h + 1) + (right_h + 1))
        return 1 + max(left_h, right_h)                    # height of THIS subtree, returned to parent
```

```
    dfs(root)
    return best[0]


def is_balanced(root):
    """Efficient O(n) single-pass check: combine height computation with balance checking.
    Returns -2 as a sentinel meaning 'already found unbalanced, stop checking further'."""
    def check(node):
        if node is None:
            return -1                                          # height of empty subtree
        left_h = check(node.left)
        if left_h == -2:
            return -2                                           # short-circuit: unbalance already found below
        right_h = check(node.right)
        if right_h == -2:
            return -2
        if abs(left_h - right_h) > 1:
            return -2                                           # THIS node violates the balance condition
        return 1 + max(left_h, right_h)                         # otherwise return true height upward
    return check(root) != -2
```

Practice Questions — Height, Depth, Diameter & Balance Checking

Q1. Maximum Depth of Binary Tree Find the height of a binary tree. *Approach:* height(node) = 1 + max(height(left), height(right)) computed bottom-up via postorder recursion; base case empty node returns -1 (or 0 depending on convention).

Q2. Minimum Depth of Binary Tree (DFS Version) Find the shortest root-to-leaf path length using DFS instead of BFS. *Approach:* Recurse into children, but a node with only ONE child must NOT be treated as a leaf — must take the min over only the existing child(ren), not blindly min(left,right) which would wrongly return 0 through a missing child.

Q3. Diameter of Binary Tree Find the length of the longest path between any two nodes. *Approach:* Compute height bottom-up while tracking a running max of left-height + right-height + 2 at every node — the path might not pass through the root at all.

Q4. Balanced Binary Tree Determine if a binary tree is height-balanced. *Approach:* Combine height computation and balance checking into a single postorder pass using a sentinel value (e.g., -2) to short-circuit as soon as any subtree is found unbalanced, avoiding $O(n^2)$.

Q5. Depth of a Specific Node Find how many edges lie between the root and a given target value. *Approach:* DFS from the root incrementing a depth counter, returning it the moment the target is found; -1 (or similar sentinel) if the target doesn't exist.

Q6. Sum of All Node Depths Compute the sum of depths of all nodes in the tree. *Approach:* A single DFS pass threading the current depth downward, accumulating depth into a running total at every node visited.

Q7. Check if Two Trees Have the Same Height Determine whether two separate trees have equal height. *Approach:* Compute height(root1) and height(root2) independently using the standard postorder height function, then compare.

Q8. Diameter of N-ary Tree Find the longest path between any two nodes in a tree with unlimited children per node. *Approach:* Generalizes binary diameter: at each node, find the TWO LARGEST child-subtree heights (not just left/right) and combine them for the through-this-node candidate diameter.

Q9. Count Nodes at Each Depth Level Return the number of nodes present at every depth level. *Approach:* DFS or BFS threading current depth, incrementing a count in a dictionary/array indexed by depth — essentially level order traversal (2.2) but counting instead of collecting values.

Q10. Height-Balanced Check with Early Termination Benchmark Compare a naive $O(n^2)$ balance checker against the O(n) single-pass version on a large skewed tree. *Approach:* Naive version recomputes height(node) from scratch for every node visited during a separate balance-check traversal, causing repeated work on skewed trees; the combined postorder version computes each height exactly once, illustrating why merging the two passes matters for performance.

2.6 Lowest Common Ancestor (LCA)

What it's for: Finds the deepest node that is an ancestor of both of two given nodes. The general binary-tree algorithm relies on recursive search; a BST's ordering property allows a much simpler and faster approach that doesn't need to search both subtrees.

Use Cases:

- Finding the closest common "ancestor" relationship in family trees, org charts, version control commit history.
- A building block for computing distance between two nodes (distance = depth(a) + depth(b) - 2*depth(LCA)).
- File system path problems (closest common parent directory).
- Answering "are these two nodes on the same branch" style queries.

Limitations:

- The general binary-tree algorithm assumes both target nodes actually exist in the tree — if one is missing, a naive implementation can return an incorrect answer rather than reporting "not found".
- The efficient BST-specific approach ONLY works because of BST ordering — applying it to a plain (non-BST) binary tree gives wrong answers.
- Repeated LCA queries on a STATIC tree are better served by preprocessing (Binary Lifting / Euler tour + Sparse Table for O(log n) or O(1) per query) rather than re-running O(n) search each time — worth knowing this exists for very query-heavy scenarios, though it's beyond this handbook's scope.

Time Complexity: O(n) for the general binary-tree algorithm (worst case visits every node); O(h) for the BST-specific approach **Space Complexity:** O(h) recursion stack

Approach (short): PSEUDOCODE (general binary tree): if root is None or root is p or root is q: return root. Recurse left and right. If BOTH recursive calls return non-null, the current root IS the LCA. Otherwise propagate up whichever side returned non-null. PSEUDOCODE (BST): starting at root, if both p and q are smaller, go left; if both larger, go right; otherwise (they split, or one equals root) the current node IS the LCA.

Keywords & Constraints: "lowest common ancestor", "LCA", "closest common ancestor of two nodes", n ≤ 10^5 nodes.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def lca_binary_tree(root, p, q):
    """General binary tree LCA -- works even without any ordering property."""
    if root is None or root is p or root is q:
        return root                                     # found one of the targets, or ran out of tree

    left = lca_binary_tree(root.left, p, q)
    right = lca_binary_tree(root.right, p, q)

    if left and right:
        return root                                      # p and q were found on DIFFERENT sides -> root is LCA
    return left if left else right                       # otherwise propagate whichever side found something


def lca_bst(root, p_val, q_val):
    """BST-specific LCA -- uses ordering to avoid searching both subtrees, O(h) instead of O(n)."""
    node = root
    while node:
        if p_val < node.val and q_val < node.val:
            node = node.left                             # both targets smaller -> LCA must be in left subtree
        elif p_val > node.val and q_val > node.val:
            node = node.right                            # both targets larger -> LCA must be in right subtree
        else:
            return node                                  # split point (or one equals node.val) -> found LCA
```

```
def distance_between_nodes(root, p_val, q_val):
    """Distance between two nodes = depth(p) + depth(q) - 2 * depth(LCA)."""
    def depth_from(node, target, d=0):
        if node is None:
            return -1
        if node.val == target:
            return d
        left = depth_from(node.left, target, d + 1)
        if left != -1:
            return left
        return depth_from(node.right, target, d + 1)

    lca = lca_bst(root, p_val, q_val)
    return depth_from(root, p_val) + depth_from(root, q_val) - 2 * depth_from(root, lca.val)
```

Practice Questions — Lowest Common Ancestor (LCA)

Q1. Lowest Common Ancestor of a Binary Tree Find the LCA of two nodes in a general (non-BST) binary tree. *Approach:* Recurse into left and right subtrees; if both sides return a non-null result, the current node is the split point (LCA), otherwise propagate up whichever side found something.

Q2. Lowest Common Ancestor of a Binary Search Tree Find the LCA of two nodes, exploiting BST ordering. *Approach:* Compare both target values against the current node: if both are smaller go left, both larger go right, otherwise the current node is exactly where the search paths diverge (or one target equals the current node) — that's the LCA.

Q3. Distance Between Two Nodes in a Binary Tree Find the number of edges between two given nodes. *Approach:* Distance = depth(p) + depth(q) - 2×depth(LCA(p,q)) — combines the LCA and depth-finding techniques directly.

Q4. Lowest Common Ancestor with Parent Pointers Find the LCA when each node has a pointer to its parent (no root access needed). *Approach:* Walk both nodes up to the root collecting ancestor chains (or use a two-pointer "meet in the middle" technique similar to finding a linked-list intersection), since parent pointers make this closer to a linked-list problem than a tree-search problem.

Q5. LCA of Deepest Leaves Find the lowest common ancestor of ALL the deepest leaves in a tree. *Approach:* A single postorder pass returns (subtree height, LCA-of-deepest-leaves-in-this-subtree) pairs; combine children's results based on whether their heights are equal (LCA is current node) or one is deeper (propagate that side's answer).

Q6. Smallest Subtree Containing All the Deepest Nodes Same problem as LCA of Deepest Leaves, framed differently. *Approach:* Identical postorder height+LCA-tracking approach — illustrates how the same underlying technique can appear under different problem phrasing.

Q7. Check if a Node is an Ancestor of Another Determine if node A is an ancestor of node B. *Approach:* Compute LCA(A, B); A is an ancestor of B if and only if LCA(A, B) is A itself.

Q8. LCA of Multiple Nodes (More Than Two) Find the LCA of a SET of k given nodes, not just two. *Approach:* Fold the general binary-tree LCA algorithm over the set: iteratively compute LCA(LCA(n1, n2), n3), and so on — pairwise LCA is associative for this purpose.

Q9. Lowest Common Ancestor in a Binary Tree III (with Parent Access, No Root) Find LCA using only parent pointers on the two nodes themselves, without ever referencing the tree root. *Approach:* Two-pointer technique: walk pointer A up from node p and pointer B up from node q; when one reaches the top, redirect it to start again from the OTHER node's starting point — they meet exactly at the LCA (same trick as linked list intersection).

Q10. Verify LCA Correctness (Preprocessing Check) Given a claimed LCA answer and two nodes, verify it's actually correct. *Approach:* Check two conditions: the claimed node must be an ancestor of BOTH p and q (path-to-root contains it), AND no descendant of the claimed node also satisfies that — reinforces precisely what "LOWEST" common ancestor means.

2.7 Path Sum & Root-to-Leaf Problems

What it's for: A family of problems that accumulate a value (usually a sum) along a root-to-leaf path, or along any downward/general path, deciding at each recursive step whether the path so far can still lead to a valid answer.

Use Cases:

- Checking if any root-to-leaf path sums to a target value.
- Enumerating ALL root-to-leaf paths (or all paths) matching a condition.
- Maximum/minimum path sum problems (may or may not require passing through the root, or being root-to-leaf).
- Counting paths (not necessarily root-to-leaf) that sum to a target — often combined with prefix-sum tricks.

Limitations:

- Easy to conflate "root-to-LEAF" (must end at a node with no children) with "root-to-ANY-node" or "any-node-to-any-node" — always re-read the exact path definition required.
- "Count paths summing to target" where paths can start/end anywhere is NOT simply solvable with basic root-to-leaf recursion alone — it needs a running prefix-sum hashmap technique (shown below) to stay O(n) instead of O(n^2).
- When enumerating (not just checking existence of) paths, must remember to pop the current node from the path list on the way back up (classic backtracking un-choose step).

Time Complexity: O(n) for existence/counting checks; O(n) with the prefix-sum technique for "any path" sum counting (vs. O(n^2) naive) **Space Complexity:** O(h) recursion stack, plus O(n) if storing all matching paths or a prefix-sum hashmap

Approach (short): PSEUDOCODE (root-to-leaf sum exists): if leaf: return remaining == node.val; else: recurse into children with remaining -= node.val, OR-ing the results together. For "any path sum" problems, carry a running prefix sum and a hashmap of {prefix_sum: count_of_occurrences} down the recursion, checking how many times (running_sum - target) has been seen before.

Keywords & Constraints: "root to leaf path sum", "path sum equals target", "count paths summing to k", n ≤ 10^5 nodes.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def has_path_sum(root, target):
    """Does any ROOT-TO-LEAF path sum exactly to target?"""
    if root is None:
        return False
    if root.left is None and root.right is None:          # leaf: check remaining target directly
        return root.val == target
    remaining = target - root.val
    return has_path_sum(root.left, remaining) or has_path_sum(root.right, remaining)


def all_root_to_leaf_paths_with_sum(root, target):
    """Return ALL root-to-leaf paths that sum to target."""
    result, current_path = [], []
    def backtrack(node, remaining):
        if node is None:
            return
        current_path.append(node.val)                       # CHOOSE: add this node to the path
        if node.left is None and node.right is None and remaining == node.val:
            result.append(current_path[:])                  # snapshot a valid complete path
        else:
            backtrack(node.left, remaining - node.val)
            backtrack(node.right, remaining - node.val)
        current_path.pop()                                  # UN-CHOOSE before returning to the parent
    backtrack(root, target)
    return result
```

```
def max_root_to_leaf_path_sum(root):
    """Maximum sum along any ROOT-TO-LEAF path."""
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return root.val
    left = max_root_to_leaf_path_sum(root.left) if root.left else float('-inf')
    right = max_root_to_leaf_path_sum(root.right) if root.right else float('-inf')
    return root.val + max(left, right)


def path_sum_iii(root, target):
    """Count paths (ANY start/end node, must go downward) that sum to target. Uses running prefix-sum + hashmap.""
    prefix_counts = {0: 1}                                  # empty prefix (sum=0) occurs once, by definition
    count = [0]

    def dfs(node, running_sum):
        if node is None:
            return
        running_sum += node.val
        # if (running_sum - target) has occurred before, that earlier point to now forms a valid path
        count[0] += prefix_counts.get(running_sum - target, 0)
        prefix_counts[running_sum] = prefix_counts.get(running_sum, 0) + 1   # CHOOSE: record this prefix sum

        dfs(node.left, running_sum)
        dfs(node.right, running_sum)

        prefix_counts[running_sum] -= 1                      # UN-CHOOSE: remove it before returning to parent
        if prefix_counts[running_sum] == 0:
            del prefix_counts[running_sum]

    dfs(root, 0)
    return count[0]
```

Practice Questions — Path Sum & Root-to-Leaf Problems

Q1. Path Sum Determine if any root-to-leaf path sums to a target. *Approach:* Subtract the current node's value from the target and recurse; a leaf succeeds only if the remaining target exactly equals its own value.

Q2. Path Sum II Return all root-to-leaf paths that sum to a target. *Approach:* Backtracking: append the current node to a running path, recurse, and pop it off before returning — record a copy of the path whenever a valid leaf is reached.

Q3. Path Sum III Count paths (any start and end, moving only downward) that sum to a target. *Approach:* Maintain a running prefix sum and a hashmap of prefix-sum counts seen along the current root-to-node path; the number of valid paths ending here equals how many times (running_sum - target) was seen before.

Q4. Maximum Root-to-Leaf Path Sum Find the maximum sum along any root-to-leaf path. *Approach:* Recurse into children, taking the max of left/right sub-results (treating a missing child as -infinity so it's never chosen), adding the current node's value.

Q5. Binary Tree Maximum Path Sum (Any Node to Any Node) Find the maximum sum path between any two nodes, not necessarily root-to-leaf. *Approach:* Each node returns its best single-direction extension upward (ignoring negative contributions via max(...,0)), while separately tracking a global best "through this node using both children" — see also DP-on-Trees in the DP handbook.

Q6. Sum Root to Leaf Numbers Each root-to-leaf path represents a number (digits along the path); sum all such numbers. *Approach:* Carry a running "number so far" downward, multiplying by 10 and adding the current digit at each step; add the accumulated number to a total whenever a leaf is reached.

Q7. Path With Maximum Minimum Value Find a path whose minimum node value along the way is as large as possible. *Approach:* Recurse returning the minimum value along the best path found so far in each subtree, combining with min(current_node.val, child_result) rather than sum.

Q8. Count Univalue Paths Count paths where every node along the path has the SAME value. *Approach:* Postorder DFS returning the length of the longest single-value path extending upward from each node, combining left+right

extensions (if both match the node's value) into a running global max/count.

Q9. Longest Path with Same Value Find the length of the longest path where all nodes share the same value (path may bend at a node). *Approach:* Same postorder-with-global-tracking pattern as Diameter (2.5), but a child's extension only counts if its value matches the current node's value; otherwise treat that side's contribution as 0.

Q10. Smallest String Starting From Leaf Find the lexicographically smallest string formed by any leaf-to-root path. *Approach:* DFS building the path's character sequence, reversing it at each leaf (since the path is being built root-to-leaf but must read leaf-to-root), and comparing candidates only at leaf nodes.

2.8 Tree Construction & Serialization

What it's for: Building a tree from other representations (traversal arrays, sorted arrays, level-order lists) and converting a tree to and from a flat, storable/transmittable string or array format.

Use Cases:

- Reconstructing a tree from preorder+inorder or postorder+inorder traversal pairs (a classic interview problem).
- Serializing a tree to save to disk, send over a network, or cache, then deserializing it back.
- Cloning a tree, converting between tree representations (e.g., array ↔ pointer-based).
- Building a BST or balanced tree directly from sorted data.

Limitations:

- Reconstructing from preorder+inorder (or postorder+inorder) REQUIRES no duplicate values unless additional information is provided — duplicates make the split point in inorder ambiguous.
- Preorder+postorder ALONE (without inorder) does not uniquely determine a general binary tree unless every node has either 0 or 2 children (full binary tree) — an important and often-missed caveat.
- A naive reconstruction that searches the inorder array for the root value each time is O(n) per node, giving $O(n^2)$ overall; using an index hashmap brings this down to O(n).

Time Complexity: O(n) with an index hashmap for construction from traversals; O(n) for serialize/deserialize
Space Complexity: O(n) for the hashmap/result structures, plus O(h) recursion stack

Approach (short): PSEUDOCODE (build from preorder+inorder): the FIRST element of preorder is always the root. Find that value's position in inorder — everything to its left is the left subtree's inorder sequence, everything to its right is the right subtree's; recursively split preorder the same way and recurse.

Keywords & Constraints: "construct binary tree from traversal", "serialize and deserialize", "build tree from array", n ≤ 10^4-10^5 nodes.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def build_tree_from_preorder_inorder(preorder, inorder):
    """Reconstruct a binary tree given its preorder and inorder traversals (assumes no duplicate values)."""
    inorder_index = {val: i for i, val in enumerate(inorder)}   # O(1) lookup instead of O(n) search
    self_preorder_idx = [0]                                      # mutable pointer into preorder, advances as we c

    def build(left, right):                                      # left/right = bounds WITHIN inorder
        if left > right:
            return None
        root_val = preorder[self_preorder_idx[0]]                # next unused preorder value IS the current root
        self_preorder_idx[0] += 1
        root = TreeNode(root_val)
        mid = inorder_index[root_val]                            # split point in inorder
        root.left = build(left, mid - 1)                         # everything left of mid = left subtree
        root.right = build(mid + 1, right)                       # everything right of mid = right subtree
        return root

    return build(0, len(inorder) - 1)


def serialize(root):
    """Serialize a tree to a comma-separated preorder string, using '#' for null children."""
    result = []
    def dfs(node):
        if node is None:
            result.append('#')
            return
        result.append(str(node.val))
        dfs(node.left)
```

```
            dfs(node.right)
        dfs(root)
        return ','.join(result)


    def deserialize(data):
        """Reconstruct a tree from the string produced by serialize()."""
        values = iter(data.split(','))
        def build():
            val = next(values)
            if val == '#':
                return None
            node = TreeNode(int(val))
            node.left = build()                                    # preorder: build left BEFORE right,
            node.right = build()                                    # exactly mirroring how it was serialized
            return node
        return build()


    def sorted_array_to_bst(nums):
        """Build a height-balanced BST from a sorted array (see also 2.3)."""
        if not nums:
            return None
        mid = len(nums) // 2
        root = TreeNode(nums[mid])
        root.left = sorted_array_to_bst(nums[:mid])
        root.right = sorted_array_to_bst(nums[mid + 1:])
        return root
```

Practice Questions — Tree Construction & Serialization

Q1. Construct Binary Tree from Preorder and Inorder Traversal Rebuild a tree given its preorder and inorder sequences. *Approach:* The first preorder value is always the root; its position in inorder splits the remaining values into left and right subtree sequences, recursed on with an index hashmap for O(1) root-position lookup.

Q2. Construct Binary Tree from Postorder and Inorder Traversal Rebuild a tree given its postorder and inorder sequences. *Approach:* Mirror of the preorder version: the LAST postorder value is the root; consume postorder from the END, building the right subtree before the left (opposite processing order).

Q3. Serialize and Deserialize a Binary Tree Convert a tree to a string and back, preserving exact structure. *Approach:* Preorder-encode the tree including explicit null markers for missing children, so deserialization can rebuild the exact structure by consuming the same sequence in the same preorder pattern.

Q4. Construct BST from Preorder Traversal Rebuild a BST given ONLY its preorder traversal (no inorder needed). *Approach:* BST ordering makes inorder redundant: recursively partition the preorder sequence using value comparisons against the root (everything less than root, in order, forms the left subtree; the rest form the right).

Q5. Convert Sorted Array to Balanced BST Build a height-balanced BST from a sorted array. *Approach:* Recursively choose the middle element as root, build left subtree from the left half and right subtree from the right half — balances automatically by construction.

Q6. Convert Sorted Linked List to Balanced BST Build a height-balanced BST from a sorted linked list. *Approach:* Use the slow/fast pointer technique to find the middle node in O(n) (see Recursion handbook, Linked Lists), make it the root, then recursively build from the two halves split at that point.

Q7. Serialize and Deserialize an N-ary Tree Convert a tree where nodes can have any number of children to a string and back. *Approach:* Encode each node’s value followed by its number of children (or an explicit end-marker) so deserialization knows exactly how many child subtrees to recursively build before returning to the parent.

Q8. Construct Binary Tree from a String with Parentheses Parse a preorder-with-parentheses string (see Tree Traversals, Q10) back into a tree. *Approach:* Recursive descent parsing: read the current value, then recursively parse a parenthesized left child (if present) and a parenthesized right child (if present) — mirrors the mutual recursion pattern from the Recursion handbook.

Q9. Clone a Binary Tree (Deep Copy) Create a completely independent deep copy of a tree. *Approach:* Preorder (or any order) recursion creating a NEW node with the same value at each step, recursively cloning left and right subtrees and attaching the clones — no serialization round-trip needed for a plain tree (no cross-links).

Q10. Find Duplicate Subtrees Find all subtrees that appear more than once in a binary tree. *Approach:* Serialize every subtree (postorder, so children's serializations are ready before the parent's) into a canonical string, count occurrences in a hashmap, and report any subtree root whose serialization count reaches exactly 2 (to report it only once).

2.9 Self-Balancing Trees (AVL) — Rotations

What it's for: A self-balancing BST that maintains, for every node, a balance factor (height of left subtree minus height of right subtree) within {-1, 0, 1} by performing local restructuring operations called **rotations** after every insertion or deletion, guaranteeing O(log n) height at all times.

Use Cases:

- Any application needing GUARANTEED O(log n) search/insert/delete, not just "usually fast" (e.g., real-time systems, database indexes).
- Situations where input arrives in a pattern that would degenerate a plain BST (already-sorted or adversarial insertion order).
- Understanding the conceptual foundation shared by other self-balancing structures (Red-Black trees, which trade slightly looser balance for cheaper rebalancing, are used in many language standard libraries like C++'s map/set).

Limitations:

- More complex to implement correctly than a plain BST — 4 distinct rotation cases (Left-Left, Right-Right, Left-Right, Right-Left) must each be identified and handled.
- Rebalancing adds constant-factor overhead to every insert/delete (extra rotations, height bookkeeping) — for read-heavy, write-rarely workloads on already-balanced data, this overhead may not be worth it over a plain BST.
- AVL trees rebalance more aggressively (strictly ≤ 1 height difference) than Red-Black trees, giving faster lookups but more rotations per write — a real tradeoff, not a strictly better structure in every scenario.

Time Complexity: O(log n) guaranteed for search, insert, and delete (height is always O(log n) by construction)
Space Complexity: O(n) for the tree, O(1) extra per rotation, O(log n) recursion stack

Approach (short): PSEUDOCODE: after a normal BST insert (recursively, bottom-up), at each ancestor on the way back up, recompute height and balance factor. If balance factor is +2 or -2, identify which of the 4 cases applies (based on the balance factor of the taller child) and perform the corresponding single or double rotation to restore balance before continuing up.

Keywords & Constraints: "AVL tree", "self-balancing BST", "rotation", "balance factor", guaranteed O(log n) requirement explicitly stated in the problem.

```
class AVLNode:
    def __init__(self, val):
        self.val = val
        self.left = self.right = None
        self.height = 1                                    # height of a new leaf is 1 (1-indexed convention here)


def get_height(node):
    return node.height if node else 0

def get_balance(node):
    return get_height(node.left) - get_height(node.right) if node else 0

def update_height(node):
    node.height = 1 + max(get_height(node.left), get_height(node.right))


def rotate_right(y):
    """Right rotation: fixes a Left-Left heavy imbalance."""
    x = y.left
    t2 = x.right
    x.right = y                                          # x becomes the new subtree root, y becomes its right
    y.left = t2                                           # y adopts x's old right subtree as its new left subt
    update_height(y)                                      # update y FIRST (it's now lower in the tree)
    update_height(x)
    return x                                               # x is the new root of this subtree

def rotate_left(x):
```

```
    """Left rotation: fixes a Right-Right heavy imbalance (mirror of rotate_right)."""
    y = x.right
    t2 = y.left
    y.left = x
    x.right = t2
    update_height(x)
    update_height(y)
    return y


def avl_insert(node, val):
    """Standard BST insert, followed by rebalancing on the way back up."""
    if node is None:
        return AVLNode(val)
    if val < node.val:
        node.left = avl_insert(node.left, val)
    elif val > node.val:
        node.right = avl_insert(node.right, val)
    else:
        return node                                          # no duplicates

    update_height(node)
    balance = get_balance(node)

    # Left-Left case
    if balance > 1 and val < node.left.val:
        return rotate_right(node)
    # Right-Right case
    if balance < -1 and val > node.right.val:
        return rotate_left(node)
    # Left-Right case: rotate left child left FIRST, then rotate node right
    if balance > 1 and val > node.left.val:
        node.left = rotate_left(node.left)
        return rotate_right(node)
    # Right-Left case: rotate right child right FIRST, then rotate node left
    if balance < -1 and val < node.right.val:
        node.right = rotate_right(node.right)
        return rotate_left(node)

    return node                                               # already balanced, no rotation needed
```

Practice Questions — Self-Balancing Trees (AVL) — Rotations

Q1. Compute Balance Factor of Every Node Given a BST, compute balance factor (left height - right height) for every node. *Approach:* Postorder traversal computing subtree heights bottom-up, recording balance = leftHeight - rightHeight at each node as it's computed.

Q2. AVL Insertion — Left-Left Case Insert a value causing a Left-Left imbalance and fix it. *Approach:* After a normal BST insert, if balance factor > 1 AND the inserted value is less than the left child's value, a single RIGHT rotation at the unbalanced node restores balance.

Q3. AVL Insertion — Right-Right Case Insert a value causing a Right-Right imbalance and fix it. *Approach:* Mirror of Left-Left: balance factor < -1 AND inserted value greater than the right child's value calls for a single LEFT rotation.

Q4. AVL Insertion — Left-Right Case Insert a value causing a Left-Right imbalance and fix it. *Approach:* A single rotation isn't enough here: first rotate the LEFT child LEFT (converting it into a Left-Left shape), then rotate the current node RIGHT — a double rotation.

Q5. AVL Insertion — Right-Left Case Insert a value causing a Right-Left imbalance and fix it. *Approach:* Mirror double rotation: rotate the RIGHT child RIGHT first, then rotate the current node LEFT.

Q6. AVL Deletion with Rebalancing Delete a node from an AVL tree and restore balance afterward. *Approach:* Perform standard BST deletion (2.4), then — unlike insertion, which needs at most one rebalance on the path — check and rebalance EVERY ancestor on the way back up, since deletion can cause imbalance at multiple levels.

Q7. Check if a Binary Tree is a Valid AVL Tree Determine if a tree satisfies the AVL balance property everywhere. *Approach:* The same efficient single-pass balance check from Balanced Binary Tree (2.5) directly answers this — AVL validity IS the balanced-binary-tree condition applied to every node.

Q8. Count Rotations Needed to Balance a BST Given an arbitrary BST, count how many rotations would be needed to make it AVL-balanced. *Approach:* Convert to a sorted array via inorder traversal, then rebuild as a perfectly balanced BST from scratch — comparing structural differences (or simply noting that a full rebuild sidesteps counting individual rotations) illustrates why rebuild-from-sorted-array is often simpler than incremental rebalancing for a badly skewed tree.

Q9. Build an AVL Tree from a Sorted Array Construct an AVL tree from sorted input directly (no rotations needed). *Approach:* Since a balanced construction (pick the middle element as root, recurse on both halves — same as 2.3/2.8) already satisfies the AVL balance property by construction, no rotations are ever triggered.

Q10. Median Maintenance Using a Self-Balancing Tree Maintain the running median of a stream of numbers using a balanced BST. *Approach:* An AVL (or any self-balancing BST) with subtree-size augmentation supports O(log n) rank/order-statistic queries, letting you find the median after every insertion without a full re-scan — demonstrates why guaranteed O(log n) height matters for streaming/online problems.

2.10 Trie (Prefix Tree)

What it's for: An N-ary tree specialized for storing strings, where each edge represents one character and every root-to-node path spells out a prefix. Unlike a BST, a Trie is NOT ordered by value comparison — it's organized by shared character prefixes, making prefix-based queries extremely fast.

Use Cases:

- Autocomplete / typeahead search — finding all words with a given prefix.
- Spell checkers, dictionary word lookups.
- IP routing (longest prefix matching), word games (Boggle, Word Search II from the Backtracking handbook).
- Any problem needing fast prefix existence checks, or counting words/prefixes sharing a common start.

Limitations:

- Space usage can be significant — each node potentially stores references for every character in the alphabet (e.g., 26 pointers for lowercase English), though a hashmap-per-node implementation mitigates this for sparse alphabets.
- Not useful for value-ordering queries (min/max/range) the way a BST is — a Trie answers "does this prefix/word exist" questions, not "what's the closest value" questions.
- Deletion is more involved than insertion: must be careful not to remove a node that's still part of another word's path (only prune nodes that have no children AND aren't the end of another word).

Time Complexity: O(L) for insert/search/prefix-check, where L is the length of the word/prefix (independent of how many words are stored!) **Space Complexity:** O(total characters across all inserted words) in the worst case (no shared prefixes); much less when prefixes are shared

Approach (short): PSEUDOCODE (insert): start at root; for each character in the word, if no child edge for that character exists, create one, then move into it; after the last character, mark that node as "end of word". Search/prefix-check follow the same character-by-character walk, differing only in whether the final node must be marked "end of word" (search) or just exist (prefix check).

Keywords & Constraints: "prefix", "autocomplete", "dictionary of words", "starts with", total input length ≤ 10^5-10^6 characters.

```
class TrieNode:
    def __init__(self):
        self.children = {}                                  # char -> TrieNode
        self.is_end_of_word = False


class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()              # create the path if it doesn't exist yet
            node = node.children[ch]
        node.is_end_of_word = True                           # mark the END of this specific word

    def search(self, word):
        """Does this EXACT word exist in the trie?"""
        node = self._walk(word)
        return node is not None and node.is_end_of_word

    def starts_with(self, prefix):
        """Does any word in the trie start with this prefix?"""
        return self._walk(prefix) is not None

    def _walk(self, s):
        node = self.root
```

```
        for ch in s:
            if ch not in node.children:
                return None                                  # path doesn't exist -> not found
            node = node.children[ch]
        return node

    def delete(self, word):
        """Remove a word, pruning nodes that become unnecessary (no children, not end of another word)."""
        def _delete(node, word, depth):
            if depth == len(word):
                if not node.is_end_of_word:
                    return False                             # word wasn't actually present
                node.is_end_of_word = False
                return len(node.children) == 0               # tell parent: safe to prune me if I have no child
            ch = word[depth]
            if ch not in node.children:
                return False
            should_prune_child = _delete(node.children[ch], word, depth + 1)
            if should_prune_child:
                del node.children[ch]
            # safe to prune THIS node too if it has no children left AND isn't the end of another word
            return len(node.children) == 0 and not node.is_end_of_word
        _delete(self.root, word, 0)
```

Practice Questions — Trie (Prefix Tree)

Q1. Implement Trie (Insert, Search, StartsWith) Build a Trie supporting word insertion, exact search, and prefix search. *Approach:* Walk/create character-by-character paths from the root for insertion; walk (without creating) for search and prefix-check, differing only in whether the final node needs is_end_of_word set.

Q2. Word Search II Find all dictionary words present in a grid of letters (see also Backtracking handbook, 2.5). *Approach:* Build a Trie from all target words, then DFS/backtrack through the grid guided by the Trie's structure — many words sharing a prefix are pruned together in a single grid traversal instead of searching for each word separately.

Q3. Longest Word in Dictionary Built One Character at a Time Find the longest word that can be built by adding one letter at a time, each prefix also being a valid word. *Approach:* Insert all words into a Trie, then DFS the Trie only continuing into children whose node is itself marked is_end_of_word — directly encodes the "every prefix must also be a word" constraint.

Q4. Replace Words (Root Replacement) Replace each word in a sentence with its shortest root found in a dictionary of roots. *Approach:* Insert all roots into a Trie; for each word in the sentence, walk the Trie character by character and stop at the FIRST is_end_of_word marker found — that's the shortest matching root.

Q5. Design Add and Search Words Data Structure Support word insertion and search where search patterns may include a wildcard '.' matching any character. *Approach:* Standard Trie insertion; search becomes a DFS/backtracking walk that, upon hitting '.', tries EVERY child edge instead of just one specific character.

Q6. Count Words with a Given Prefix Given a list of words, count how many start with a specific prefix. *Approach:* Augment each Trie node with a count of how many words pass through it during insertion; a prefix query then just walks to that node and reads its count directly, avoiding a rescan of all words.

Q7. Maximum XOR of Two Numbers in an Array Find the maximum XOR value between any two numbers in an array (Trie over BITS instead of characters). *Approach:* Build a binary Trie where each number is inserted bit by bit (MSB to LSB); for each number, greedily walk the Trie preferring the OPPOSITE bit at each level to maximize the XOR result — shows Tries generalize beyond string characters to any fixed-length symbol sequence.

Q8. Palindrome Pairs Find all pairs of words in a list that concatenate to form a palindrome. *Approach:* Insert reversed words into a Trie augmented with palindrome-suffix markers, then for each word check the Trie for complementary reversed prefixes/suffixes that would complete a palindrome — an advanced Trie application combining string and palindrome techniques.

Q9. Implement Magic Dictionary (One Character Different) Support search queries that match a dictionary word with EXACTLY one character changed. *Approach:* Standard Trie search, but at each character position also try recursively continuing with every OTHER possible character (using up a "one edit allowed" budget) besides the exact match, backtracking if the budget is exhausted early.

Q10. Stream of Characters (Suffix Matching) Given a stream of characters, determine after each character whether any suffix of the stream so far matches a dictionary word. *Approach:* Build a Trie from the REVERSED dictionary words; maintain a running buffer of recent stream characters and check the reversed buffer against the Trie after each new character, since matching a suffix of the stream is equivalent to matching a prefix of the reversed stream.

PART 3: QUICK REFERENCE CHEAT SHEET

Topic	Purpose	Time	Space	Fails When
Traversals (Pre/In/Post)	Visit every node in a defined order	O(n)	O(h)	Skewed tree → O(n) recursion depth
Level Order (BFS)	Visit level by level	O(n)	O(w)	Wide tree → O(n) queue space
BST Insert/Search	Maintain ordered structure	O(h)	O(h)	Unbalanced BST degrades to O(n)
BST Deletion	Remove while preserving order	O(h)	O(h)	Two-children case mishandled -> broken BST
Height/Diameter/Balance	Measure tree shape	O(n)	O(h)	Naive re-computation → $O(n^2)$
LCA	Find deepest common ancestor	O(n) general / O(h) BST	O(h)	Using BST trick on a non-BST tree
Path Sum Problems	Accumulate along paths	O(n)	O(h) or O(n)	"Any path" sums need prefix-sum hashmap, not naive DFS
Construction/Serialization	Build/store/restore a tree	O(n)	O(n)	Duplicate values break traversal-pair reconstruction
AVL / Self-Balancing	Guarantee O(log n) height	O(log n)	O(n)	Wrong rotation case identified
Trie	Fast prefix operations	O(L)	O(total chars)	Used for value-ordering queries instead of prefixes

Decision Guide — “Which Pattern Do I Use?”

1. Need to visit every node in a defined depth-first order (root-first, sorted, or children-first)? → Traversals (Preorder/Inorder/Postorder)

2. Need results grouped by depth, or the shortest root-to-leaf path? → Level Order Traversal (BFS)

3. Need to add a value while keeping the tree ordered? → BST Insertion

4. Need to remove a value while keeping the tree valid? → BST Deletion

5. Need to measure the tree’s shape (how tall, how balanced, longest path)? → Height/Diameter/Balance Checking

6. Need the closest shared ancestor of two nodes? → Lowest Common Ancestor

7. Need to accumulate a value (usually sum) along paths and check/count/enumerate? → Path Sum Problems

8. Have a traversal (or sorted array) and need to REBUILD the tree, or need to save/restore it? → Tree Construction & Serialization

9. Need GUARANTEED O(log n) operations regardless of insertion order? → Self-Balancing Trees (AVL)

10. Working with STRINGS and need fast prefix lookups/autocomplete? → Trie